HistDB: Historical Bug Knowledge-Guided Testing of Cypher-Compatible Graph Databases

ANONYMOUS AUTHOR(S)

Historical bug reports provide evidence of query behavior associated with real implementation failures, but are rarely reused to guide test generation. Replaying historical triggers preserves context but largely revisits known behavior, while unconstrained query-fragment composition can violate dependencies or semantic coherence. We present HistDB, a historical bug knowledge-guided testing framework for Cypher-compatible graph database management systems (GDBMSs). HistDB decomposes historical triggers into parameterized seed skeletons that preserve structural context and source-linked semantic features extracted from associated query evidence. Given a seed, it selects structurally supported features from a global historical pool, fixes the composition target before generation, and prompts an LLM to construct setup and main queries. We evaluate HistDB on Neo4j, Memgraph, FalkorDB, Apache AGE, and ArcadeDB against six testing approaches. Compared with within-report reuse, HistDB generates 49.7% more distinct historically unseen query-element pairs and increases the proportion of queries containing such pairs by 8.62 percentage points. In the multi-version campaign, it yields 148 manually validated bug reports, including 109 logic bugs; developers confirmed 123 reports, and 99 had documented fixes. In 24-hour comparisons, HistDB identifies 50 validated bugs, 48 of which no baseline finds, and achieves the highest observed line coverage on Neo4j and Memgraph. Separate query-diversity runs show 53.4% more distinct query-element pairs and 64.3% more core pairs than GRASpDB, although GRASpDB finds one additional bug missed by HistDB. Overall, HistDB enables broader exploration of unseen semantic combinations and effective bug discovery while improving query diversity and line coverage on instrumented systems.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Information systems** → *Graph-based database models.*

Additional Key Words and Phrases: graph database testing, Cypher, historical bugs, test generation, large language models, differential testing, metamorphic testing

ACM Reference Format:
Anonymous Author(s). 2027. HistDB: Historical Bug Knowledge-Guided Testing of Cypher-Compatible Graph Databases. *Proc. ACM Program. Lang.* 11, OOPSLA (April 2027), 26 pages.

1 Introduction

Cypher-compatible graph database management systems (GDBMSs) support applications ranging from knowledge graphs and recommendation systems to social-network analysis and software engineering. Testing these systems is challenging because a single Cypher query [1] can combine path traversal, variable binding, aggregation, subqueries, null handling, and graph updates, whose behavior depends jointly on graph topology, variable scope, and intermediate results. As a result, many defects arise from interactions among multiple semantic features rather than from isolated language constructs. The central challenge is therefore not merely to generate more valid queries, but to direct limited testing effort toward semantic interactions that are both executable and likely to expose real implementation faults.

Existing GDBMS testing techniques have substantially improved automated test generation and oracle construction. GDsmith, for example, exploits grammar and database-state constraints to generate valid graph queries [2], while Dinkel maintains query context and graph-schema state to support complex Cypher queries [6]. DGDB uses large language models (LLMs) for graph-query generation and differential testing [7]. These approaches provide increasingly effective ways to

2027. ACM 2475-1421/2027/4-ART
https://doi.org/

construct executable tests and, in some cases, identify anomalous executions. However, their search is largely driven by syntax, the current database state, model priors, or predefined transformations. The failure experience accumulated during the development and maintenance of real GDBMSs is rarely used as an explicit source of guidance. Consequently, large regions of the semantic search space remain explored without direct evidence about which interactions have previously exposed implementation faults.

Historical bug reports provide such evidence. A confirmed bug report records a query that actually exercised an incorrect implementation behavior and the execution context in which the failure became observable. This makes historical failures a potentially valuable source of testing guidance. Yet existing ways of reusing such artifacts are inherently limited. Replaying a historical trigger preserves the context of the original failure but largely revisits behavior that has already been observed. Generic generation, in contrast, can explore novel tests but provides little basis for prioritizing combinations that are empirically associated with real failures. What is missing is a principled way to use historical failures not only as regression inputs, but also as evidence for exploring new query interactions that may expose previously unknown failures.

We therefore propose a different view of historical bug reports: **historical failures can serve as compositional testing knowledge**. The key idea is to separate what must be preserved from what can be recombined in a principled manner. A historical trigger provides structural context that constrains how query behaviors can be realized, while historically observed, source-linked semantic features provide reusable testing cues that can be combined across reports. The resulting test is not intended to reproduce any historical query verbatim. Instead, HistDB aims to generate a new combination of historically observed features while using the seed skeleton as structural guidance to retain the relevant execution context.

This perspective gives rise to a concrete methodological challenge: how can historical failure knowledge be represented so that it supports novel combinations without losing the structural and state dependencies required for execution? HistDB addresses this challenge through a historical bug knowledge-guided testing framework for Cypher-compatible GDBMSs. HistDB converts eligible historical triggers into reusable seed skeletons and source-linked semantic features. Given a seed skeleton, it identifies historical features that have structural support in that context and fixes the resulting composition target before generation. An LLM is then prompted with this explicit testing intent to jointly construct setup queries and a main query intended to realize the selected interaction. The resulting test is executed from a clean state, and anomalous cases are reproduced, minimized, and validated before reporting.

The separation between historical knowledge and test construction is important. HistDB does not ask the LLM to discover which historical behaviors are worth combining. Instead, the composition target is determined explicitly by the historical corpus and the seed structure, making the intended reuse auditable and independent of any particular model. This design turns historical failure experience into a search signal for exploring semantic combinations that need not have co-occurred in any original report.

We evaluate HistDB on five independently implemented Cypher-compatible GDBMSs (Neo4j, Memgraph, FalkorDB, ArcadeDB, and Apache AGE) and compare it with six representative testing approaches. In the controlled generalization study, cross-report composition yields 49.7% more distinct historically unseen query-element pairs than within-report reuse, while increasing the proportion of queries containing such pairs by 8.62 percentage points. The multi-version bug-reporting campaign produces 148 manually validated bug reports, 109 of which are logic bugs; 123 reports were confirmed by developers, and 99 had documented fixes. Under a 24-hour testing budget, HistDB discovers 50 validated bugs, including 48 that none of the baselines detects. It also achieves the highest observed line coverage on Neo4j and Memgraph, the two systems supporting

source-line instrumentation. Further analysis of query diversity shows that HistDB generates 53.4% more distinct query-element pairs and 64.3% more core pairs than GRASpDB, the strongest baseline on both measures. Nevertheless, GRASpDB identifies one additional bug that HistDB misses. Overall, these findings demonstrate how historical bug knowledge can guide test generation beyond conventional regression replay.

This paper makes three contributions:

- We identify and formulate the problem of *compositional reuse of historical failure knowledge* for automated GDBMS testing, and show why conventional replay or unconstrained generation does not adequately address it.
- We present HistDB, which separates historical triggers into structure-preserving seed skeletons and historically observed, source-linked semantic features, selects features in the context of a seed skeleton, and jointly constructs executable tests with the required graph state.
- We comprehensively evaluate HistDB across diverse Cypher-compatible GDBMSs and against existing testing approaches, providing empirical evidence of its effectiveness in generalizing historical knowledge to unseen query-element combinations, discovering validated bugs, and improving query diversity and line coverage on instrumented systems.

The remainder of this paper is organized as follows. Section 2 establishes the basis for treating historical failures as composable testing knowledge. Section 3 presents the HistDB approach. Section 4 evaluates HistDB, Section 5 discusses the results, and Section 6 reviews related work. Finally, Section 7 concludes the paper.

2 Historical Failures as Composable Testing Knowledge

Historical bug reports are commonly preserved as regression artifacts: they record inputs that exposed an implementation failure and can be replayed after changes. We argue that these reports contain a broader form of reusable knowledge. A historical trigger captures both *structural context*, such as clause organization and variable reuse, and *historically observed, source-linked semantic features*, such as subquery updates and scope transfer. These two forms of knowledge play different roles in test construction: structure determines where and how a feature can be realized, whereas semantic features guide which query behaviors a generated test explores.

This distinction exposes a limitation of replay-centric reuse. Replaying a historical trigger preserves its dependencies but confines exploration to behaviors that already co-occurred in that report. At the other extreme, freely mixing query fragments can cross report boundaries but may violate variable dependencies or graph-state requirements. Unconstrained generation can produce diverse queries, yet it lacks a direct basis for prioritizing query behaviors grounded in historical failure evidence. These limitations motivate a *compositional view of historical failures*: rather than treating historical reports as indivisible regression cases, testing can preserve structural context as reusable seed skeletons while representing historically observed query behaviors as source-linked semantic features that can be recombined across reports.

The remainder of this section develops this perspective in three steps. Section 2.1 identifies the limitations of replay-centric reuse and formulates the resulting testing problem. Section 2.2 introduces the structural–behavioral decomposition that makes cross-report composition possible. Section 2.3 uses a developer-confirmed Memgraph bug as a motivating example and derives the design principles that guide the approach presented in Section 3.

2.1 The Limits of Replay-Centric Reuse

Testing Cypher-compatible GDBMSs is inherently state- and context-dependent. Pattern matching produces bindings and rows; `WITH` and subqueries determine which bindings remain visible;

aggregation changes row multiplicity; and updates alter the graph state observed by subsequent clauses [8, 9]. Consequently, a useful test must do more than contain syntactically valid Cypher: it must establish a graph state and a sequence of bindings in which the intended interaction becomes executable and observable. A generated test can therefore be organized as an ordered set of setup queries followed by one main query. The setup establishes the required graph state, while the main query exercises the target interaction and exposes its effect through returned rows or execution status.

Historical failures provide valuable evidence for such interactions, but existing forms of reuse provide only partial support for exploration.

Whole-trigger replay preserves dependencies but preserves too little exploration. Reusing a complete historical trigger keeps its clause organization, variable bindings, and data assumptions intact, making the resulting test highly faithful to the original failure. However, all semantic features remain tied to their original co-occurrence. Replay can reproduce known behavior, but it cannot directly explore combinations that were absent from the historical report. Even template-based adaptation, which replaces application-specific values while preserving the original structure, remains centered on a single historical configuration.

Literal fragment mixing expands the search space but discards essential context. Fragments taken from different reports can introduce new combinations, but query features are not independent pieces of syntax. A fragment may depend on variables bound earlier in the query, on a particular clause ordering, on the presence of a subquery or aggregation boundary, or on a specific graph topology. Literal concatenation can therefore yield invalid or semantically incoherent tests. The problem is not merely syntactic compatibility: the selected features must be realizable together in a coherent execution context.

Unconstrained generation provides diversity but lacks historical-failure guidance. An LLM or a generic query generator can explore a large space of valid Cypher programs, but the generated combinations are primarily determined by model priors, grammar constraints, or the current database state. Such generation does not explicitly exploit historically observed, source-linked semantic features as testing cues derived from bug reports. It therefore lacks a principled way to guide exploration using historical failure evidence.

These observations lead to the central problem addressed in this work:

> **Given a corpus of historical failure triggers, how can we construct executable tests by combining historically observed, source-linked semantic features across reports while preserving the structural context and graph-state dependencies for realizing them?**

This problem differs from regression reproduction. The goal is not to reconstruct a historical trigger T_i, but to construct a new test whose semantic features may be distributed across multiple historical reports. Let a historical trigger be represented conceptually as

$$T_i = (S_i, B_i) \tag{1}$$

where S_i denotes the reusable structural context and B_i denotes the historically observed, source-linked semantic features for trigger i. Replay operates on the original pair (S_i, B_i), whereas the compositional approach developed in Section 3 instead seeks compositions of the form

$$T' = (S_i, \{b_{j_1}, b_{j_2}, \ldots, b_{j_k}\}), \tag{2}$$

where the selected features may originate from different historical reports and need not have previously co-occurred. The composition is useful only when the selected features remain realizable within the structural context and graph state of the generated test.

This formulation shifts the role of historical bugs from *fixed regression inputs* to *a searchable space of reusable failure experience*. The key question is therefore not whether historical triggers can be replayed, but how their reusable information should be represented so that new combinations can be explored without destroying the execution context that makes those features meaningful.

2.2 A Structural–Behavioral Decomposition

A historical Cypher trigger contains two complementary forms of reusable knowledge. The first is *structural knowledge*: the organization that determines how an interaction is embedded in a query. It includes clause ordering, repeated variable occurrences, path bindings, scope boundaries, and read/write sites. Such structure often remains relevant across applications even when the original labels, property names, relationship types, and literal values do not. We therefore represent this information as a *seed skeleton*: a parameterized representation that preserves the organization and binding relationships required to realize an interaction while abstracting away application-specific values. The second is *behavioral knowledge*: query-level semantic patterns and interactions observed in the query evidence associated with a historical bug report. Examples include performing a write inside a `CALL` subquery, propagating variables across a scope boundary, traversing a particular path form, or combining reads and updates in a particular sequence. We refer to these as *historically observed, source-linked semantic features*. Each feature describes a reusable query behavior independently of the concrete variable names and values in its source query, while retaining provenance to the historical report and query from which it was extracted.

The distinction is important because structure and behavior answer different questions: the *skeleton* specifies *where and how* a mechanism can be realized, whereas the *semantic features* specify *what behavior* the generated test should realize. Neither representation is sufficient on its own. A skeleton without reusable features remains close to replay. Features without structural context reduce the query to a collection of behavioral labels with no explicit organization for realizing them. The separation therefore creates a composition interface between historical cases: a skeleton can provide the execution context, while features from other reports can be selected and instantiated within that context.

This interface also makes clear why cross-report composition cannot be treated as unrestricted feature mixing. A semantic mechanism is useful only when the chosen structural context provides a plausible way to realize it. For example, a subquery-related mechanism may require an existing `CALL` boundary, while a path-related mechanism may require an appropriate graph topology; scope-sensitive behavior may depend on bindings surviving earlier `WITH` clauses. Compatibility is therefore *structural rather than purely lexical*. In a compositional approach, the seed skeleton provides the context in which historical features are selected and subsequently realized.

Conceptually, the resulting composition can be viewed as a constrained mapping

$$(S_i, \mathcal{B}) \rightarrow T, \tag{3}$$

where S_i is a structural context, $\mathcal{B}$ is a set of selected historical features, and T is an executable setup–query test. The construction is expected to exhibit three properties: the structural organization of S_i is preserved, the selected features are realized in the generated query, and the setup provides the graph state required for that realization. The resulting test can then be evaluated using the applicable execution oracle and, when an anomaly is found, reduced and validated.

This decomposition also clarifies the role of an LLM in compositional testing. Historical evidence and structural compatibility should guide the selection of semantic features, while a generation model can instantiate the selected features as a coherent executable test. Separating feature selection from test realization makes the use of historical knowledge auditable and avoids tying the approach to any particular generation model.

2.3 A Motivating Example and Design Principles

Fig. 1 illustrates how knowledge from historical bug reports can be recombined to construct a new test. The seed skeleton comes from Neo4j issue #12657 [39]. It preserves a sequence of `MATCH`, `OPTIONAL MATCH`, `WITH ... collect(...)`, and `RETURN` clauses, providing a structural context for new behaviors while abstracting application-specific details. Three features are selected from other reports: `RETURN` projection aliasing (F1) from ArcadeDB issue #3560 [45], a pattern-comprehension mechanism (F2) from Neo4j issue #13221 [40], and ordering with a limit (F3) from RedisGraph issue #2386 [65]. These features originate from separate reports and are combined within the seed-derived structure rather than copied as complete query fragments.

The generated test uses a graph containing an A node with name = 'Alice' and age = 30. This node has no outgoing KNOWS or FRIEND relationships; the remaining disconnected graph components are omitted from the figure. The main query preserves the seed-derived matching and aggregation structure, uses a pattern comprehension in `ORDER BY`, applies `LIMIT 3`, and returns aliased node properties. The query returns Alice, age 30, and an empty friends list on Neo4j 2026.04.0, but raises the error Pattern Comprehension expected a list, got null on Memgraph 3.10.1.

The focused controls shown in Fig. 1 help isolate the interaction behind this discrepancy. Evaluating the pattern comprehension in `RETURN` succeeds, as does using `OPTIONAL MATCH` with `collect(...)` without the pattern-comprehension ordering. Combining these constructs through `ORDER BY size([...])`, however, triggers the error. The reduced query further retains the pattern-comprehension ordering while removing `LIMIT 3` and the final ORDER BY age; F3* in the figure denotes the ordering component of F3 retained after reduction. The resulting case shows that the failure does not require either removed clause. Memgraph issue #4180 [43] documents the related failure and controls, and PR #4487 [44] fixed the issue.

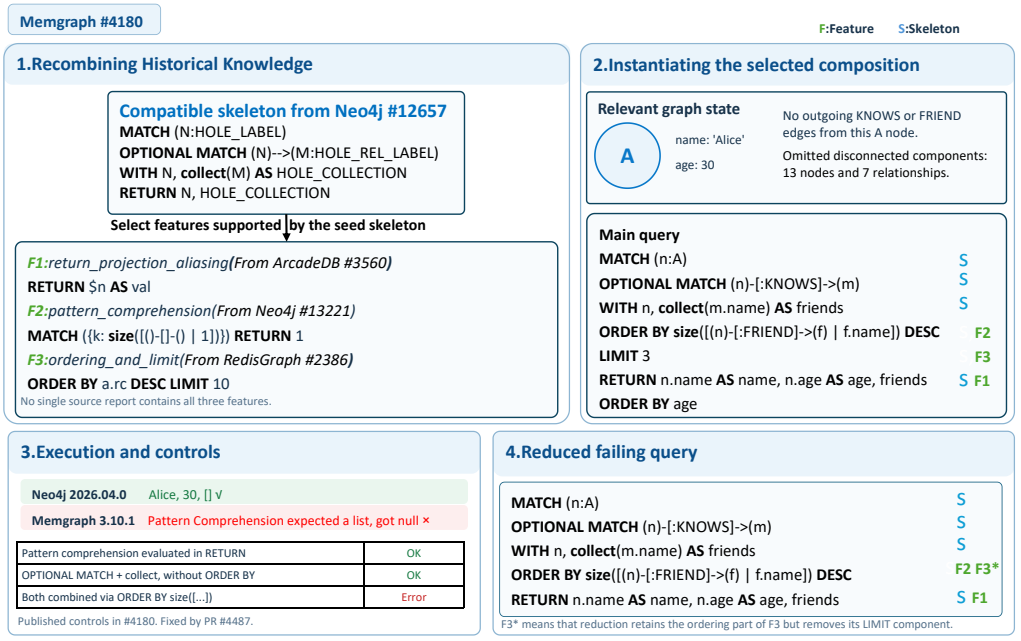


Fig. 1. Motivating example. A seed skeleton combined with three semantic features from different reports produces a query that succeeds on Neo4j but triggers a pattern-comprehension error on Memgraph.

The example suggests three design principles for **compositional testing with historical failure knowledge**.

P1: Preserve the structure needed to realize a behavior. Historical abstraction should remove application-specific details while retaining the query context that constrains execution, including clause organization, variable bindings and reuse, scope boundaries, and the placement of relevant read/write or aggregation operations. The resulting skeleton should preserve enough structural context to support new feature instantiations without simply reproducing the original query.

P2: Select historical features in structural context. Cross-report composition should not amount to arbitrary mixing of query fragments. The seed skeleton provides evidence about where a feature may have a plausible realization opportunity, while source provenance retains its connection to the historical report from which it was derived. This context should guide feature selection, but structural plausibility alone does not guarantee that independently selected features are mutually compatible.

P3: Construct the graph state and query jointly, then validate observed anomalies. Selected features alone do not constitute an executable test. The graph state and query must be constructed together to establish the data conditions, variable bindings, and other semantic preconditions needed to expose the intended interaction. The test should be executed from a clean state, with focused controls and failure-preserving reduction used to assess whether the observed anomaly is reproducible and attributable to the intended interaction.

These principles guide the approach in Section 3, with P1 informing structure-preserving historical representation, P2 guiding skeleton-conditioned feature selection, and P3 motivating joint test construction and failure-preserving validation.

3 The HistDB Approach

3.1 Overview

Given a historical corpus and a target GDBMS, HistDB operationalizes the three principles established in Section 2 by separating historical triggers into structure-preserving skeletons and historically observed, source-linked semantic features, selecting features in the context of a seed skeleton, and jointly constructing and validating executable tests.

Fig. 2 organizes the approach into four stages. First, *historical-failure representation* converts eligible historical triggers into separate skeleton and feature records, producing the frozen historical

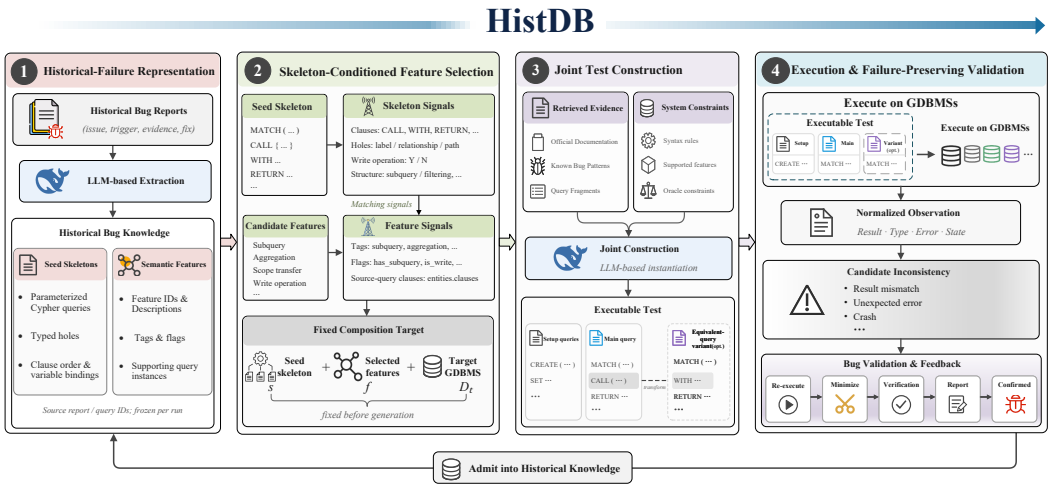


Fig. 2. HistDB workflow for historical-knowledge-guided test construction and validation.

knowledge used by subsequent stages. Second, *skeleton-conditioned feature selection* samples a seed skeleton and selects historical features that have structural support in that context. The resulting composition target is fixed before LLM-based construction begins. Third, *joint test construction* uses the seed skeleton, selected features, retrieved supporting evidence, and target-system constraints to guide the generation of setup queries and a main query intended to realize the target interaction. Finally, *execution and failure validation* executes the generated test from a clean state, evaluates its behavior using applicable oracles, reproduces oracle violations, reduces reproducible failures, and forwards candidates for manual validation.

Each stage produces an explicit artifact that serves as input to the next stage: a frozen skeleton and feature pool; a selected seed skeleton and feature set together with the target GDBMS; a test artifact comprising ordered setup queries, one main query, and an optional equivalent-query variant; and an execution record supporting reproduction, reduction, and review. Importantly, the composition target is determined before generation. The LLM therefore instantiates an explicit testing objective rather than deciding which historical knowledge to reuse.

Algorithm 1 summarizes HistDB's end-to-end control flow. Lines 1–2 construct the historical knowledge representation and retrieval index. Lines 5–8 select a seed skeleton and historically observed, source-linked semantic features, fix the composition target, and retrieve context. Lines 9–15 perform bounded construction with a fixed target across retries and check basic output admissibility before execution. Finally, Lines 16–24 execute oracles, reproduce violations, reduce reproducible failures, and retain bug reports only after manual validation. Sections 3.2–3.5 explain these stages, while Section 3.6 describes implementation and reproducibility support.

3.2 Historical-Failure Representation

HistDB operationalizes the structural–behavioral decomposition in Section 2.2 by converting each eligible historical trigger into one parameterized skeleton record and one or more semantic-feature records. A report is eligible when its triggering query can be recovered, its target GDBMS and version are known, and the available evidence is sufficient to establish both the observed and expected behavior. The original graph state is not required: when setup queries are unavailable, compatible state can be synthesized during joint test construction. Each retained report records its target system and version, triggering query, available setup and behavioral evidence, and source report and query identifiers for provenance.

Skeleton records. A skeleton record stores parameterized Cypher text, typed holes, and source identifiers. Parameterization abstracts application-specific labels, relationship types, property keys, and literals while preserving clause order, repeated variable references, and read/write sites. HistDB also derives lightweight structural descriptors, such as clause kinds and hole types, which guide subsequent feature selection.

Semantic-feature records. A feature record represents a reusable query behavior observed in a historical trigger query. It stores a stable ID, canonical name, natural-language description, tags, Boolean structural flags, supporting query instances, and source identifiers. Tags capture constructs such as `call`, `subquery`, and `write`, while flags enable uniform checks such as `has_subquery=true` and `is_write=true`. Supporting instances provide concrete historical evidence but are not copied as templates. For example, `call_subquery_with_create` denotes a write inside a `CALL` subquery that can be instantiated with different variables and relationship types.

HistDB uses two separate schema-constrained LLM calls to parameterize triggering queries and extract testing-relevant semantic features. The pipeline attaches metadata and provenance rather than asking the model to infer them. Algorithm 1, Lines 1–2, implement this stage: RepresentHistoricalFailures constructs the skeleton and feature pools, and BuildRetrievalIndex indexes the historical reports, extracted records, and auxiliary retrieval corpus for subsequent

Algorithm 1: **HistDB**

Input : Historical reports H; target GDBMS D_t; available comparison systems $\mathcal{D}_{\text{cmp}}$; auxiliary retrieval corpus A; generation-attempt budget N; feature count m; retry bound R.
Output : Manually validated bug reports F.

$(S, P) \leftarrow \textsc{RepresentHistoricalFailures}(H)$
$I \leftarrow \textsc{BuildRetrievalIndex}(H, S, P, A)$ *// Frozen for this run*
$F \leftarrow \emptyset$
for $i \leftarrow 1$ **to** N **do**
  $s \leftarrow \textsc{SampleSkeleton}(S)$
  $\Phi \leftarrow \textsc{SelectFeatures}(s, P, m)$
  $O \leftarrow (s, \Phi, D_t)$ *// Fixed composition target*
  $C \leftarrow \textsc{Retrieve}(I, O, D_t)$
  $ok \leftarrow \mathit{false}$
  for $j \leftarrow 1$ **to** R **do**
    $T \leftarrow \textsc{JointConstruct}(O, C, D_t)$ *// $T = (U, q, v_{\text{eq}}?)$*
    $(ok, \rho) \leftarrow \textsc{PreExecutionCheck}(T, D_t)$ *// Check basic output admissibility*
    if ok **then**
      break
    $C \leftarrow \textsc{AugmentContext}(C, \rho)$ *// O remains unchanged*
  if ok **then**
    $\Omega \leftarrow \textsc{SelectApplicableOracles}(T, D_t, \mathcal{D}_{\text{cmp}})$
    $E \leftarrow \textsc{ExecuteAndObserve}(T, \Omega)$
    if $\textsc{OracleViolation}(E, \Omega)$ **then**
      $(rep, E_{\text{rep}}) \leftarrow \textsc{Reproduce}(T, E, \Omega)$
      if rep **then**
        $(T', E') \leftarrow \textsc{FailurePreservingReduce}(T, E_{\text{rep}}, \Omega)$
        $f \leftarrow \textsc{ManualValidateAndReport}(T', E', D_t, \Omega)$
        if $f \neq \emptyset$ **then** $F \leftarrow F \cup \{f\}$
return F

retrieval. The pools and index remain frozen throughout each run; newly validated bug reports affect feature selection and retrieval only in a subsequent corpus snapshot.

3.3 Skeleton-Conditioned Feature Selection

Given a seed skeleton, HistDB selects historically observed semantic features from the global historical feature pool. Structural descriptors provide a soft prior that favors features with plausible realization sites in the seed skeleton, while allowing features from different historical reports. Let m denote the requested number of distinct features per composition.

Cross-report provenance. HistDB classifies a composition as *cross-report* only if no single retained report contains all selected feature IDs. Different immediate source reports are insufficient to establish cross-report composition because the complete feature set may already co-occur in another report. Source report and query identifiers make this condition checkable before generation.

Skeleton–feature structural support. Let k denote a seed skeleton and f a candidate feature. HistDB computes their structural support as

$$a(k, f) = \sum_{r \in \mathcal{R}} c_r(k, f), \tag{4}$$

where $\mathcal{R}$ is the fixed set of structural matching rules and $c_r(k, f)$ is the score contributed by rule r. Table 1 summarizes the rules, which compare skeleton clauses and typed holes with feature tags, source-query clauses, and behavioral flags.

The scores encode a hierarchy of structural evidence rather than semantic compatibility or failure probability. Strong correspondence receives +6 when the skeleton directly provides a realization site, such as a CALL clause for a subquery feature or a write clause for a write feature. Explicit syntactic correspondence receives +4 to 5, while weaker contextual signals receive +1–2. The weights are fixed before evaluation and are neither learned nor tuned against evaluation outcomes.

Candidate pool and feature sampling. Given a seed skeleton k, we first identify the features with positive structural support:

$$E(k) = \{f \in P \mid a(k, f) > 0\}. \tag{5}$$

When $E(k)$ is nonempty, it serves as the sampling pool. Otherwise, HistDB falls back to the full feature pool P. From the resulting pool, HistDB samples up to m distinct features without replacement. Let $E_t(k)$ denote the candidates remaining before the t-th draw, initialized to the chosen sampling pool. The probability of selecting $f \in E_t(k)$ is given by:

$$p_t(f \mid k) = \frac{a(k, f) + 1}{\sum_{g \in E_t(k)} (a(k, g) + 1)}. \tag{6}$$

The additive constant gives every remaining candidate a nonzero selection probability. If all candidates have zero structural support, sampling becomes uniform. Otherwise, features with stronger support are more likely to be selected, although none is guaranteed.

For example, a skeleton containing `MATCH`, `CALL`, `WITH`, and `RETURN` but no write clause or path-related hole gives a subquery feature stronger structural support than a variable-length-path feature. The former is therefore more likely to be selected, without being forced into the composition.

Importantly, $a(k, f)$ evaluates each skeleton–feature pair independently and does not model interactions among multiple selected features. It is therefore a selection heuristic, not a semantic-compatibility oracle. The selected features guide joint query construction, while subsequent execution validation assesses the resulting query's behavior.

Table 1. Heuristic rules and weights for estimating structural support between skeletons and features.

Evidence level	Skeleton signal	Feature signal	Score
Strong	Write clause	write tag/flag	+6
	`CALL`	subquery flag/tag	+6
	Path-related hole	path-pattern flag/tag	+6
	`OPTIONAL MATCH`	optional-match flag/tag	+6
Explicit	Shared clause	same clause in tags or `entities.clauses`	+4 per field
	`ORDER BY`	ordering flag/tag	+5
	`SKIP` or `LIMIT`	pagination flag	+4
Contextual	No write clause	write flag false or absent	+2
	`CASE`	CASE flag/tag	+1
	`WHERE`	predicate or where tag	+2
	Path-related hole	pattern-comprehension flag/tag	+2
	Label/relationship hole	match or merge tag	+1
	`RETURN` or `WITH` aggregation	aggregation flag/tag	+2

Algorithm 1, Lines 5–7, implement this stage. Line 5 samples the seed skeleton, Line 6 selects up to m distinct features using the structural-support scores defined above, and Line 7 fixes the composition target $O = (s, \Phi, D_t)$. This target remains unchanged during subsequent retrieval and construction retries. The selection score guides which features are chosen but does not guarantee that they can all be realized together in a valid query.

3.4 Joint Test Construction

Once the composition target is fixed in Section 3.3, HistDB attempts to construct an executable test conditioned on the selected skeleton, features, and target GDBMS; these inputs remain fixed across retrieval and construction retries. This stage comprises three steps: retrieving supporting context, jointly constructing the setup and main query, and checking the candidate before execution.

Algorithm 1, Lines 8–15, implement this stage. HistDB retrieves supporting context, constructs a candidate test, and checks its validity before execution. When a candidate fails the pre-execution checks, the rejection reasons guide a bounded reconstruction process.

Retrieving supporting context. HistDB builds a retrieval request from the seed skeleton's structural descriptors, the selected features' names, descriptions, and tags, and the target-system constraints. It retrieves relevant examples from historical queries, query clauses, skeletons, feature instances, and available official documentation. Retrieved items are organized by their roles, such as structure, predicates, aggregation, ordering, and updates. They provide construction evidence rather than new testing objectives: retrieval neither changes the composition target nor adds features to the selected feature set. This step corresponds to Algorithm 1, Line 8.

Joint test construction. HistDB asks the LLM to produce, in one schema-constrained response, an ordered list of setup queries and one main query. The seed skeleton supplies clause organization, repeated variable references, typed holes, and read/write sites; selected features specify the behaviors to realize; retrieved context provides syntax and usage examples. The LLM is prompted to construct a test for the fixed objective rather than choose which historical features to combine.

The resulting test artifact is $T = (U, q, v_{\text{eq}}?)$, where U is the ordered list of setup queries, q is the main query, and v_{eq} is an optional equivalent-query variant when an applicable transformation is available. Setup queries are intended to establish the graph state required by the main query, which is intended to establish variable bindings and combine selected behaviors within the seed-derived structure. Labels, relationship types, property names, and literals are generated to maintain consistency across setup and main queries. When executable setup is unavailable in the source report, HistDB synthesizes compatible setup queries from the main query's dependencies.

Generating an equivalent-query variant does not by itself establish semantic equivalence. As described in Section 3.5, the corresponding oracle is applicable only when the transformation's preconditions hold and its required semantic properties are preserved.

Pre-execution record checks. Before execution, HistDB checks that the main query is nonempty, begins with a recognized Cypher entry form, and contains no unresolved holes. It also checks every setup query for unresolved holes; optional target-specific filters may reject disallowed constructs. Algorithm 1, Line 12, performs these checks within the bounded retry loop of Lines 10–15. If a candidate passes, Lines 13–14 exit the loop. Otherwise, Line 15 uses the rejection reasons ρ to augment the construction context C for another attempt.

Throughout these retries, the composition target $O = (s, \Phi, D_t)$ remains unchanged; only the construction context may change. If no candidate passes within the retry bound R, the candidate is discarded rather than executed under a modified target. Candidates that pass are forwarded to Section 3.5 for execution and oracle-based validation.

3.5 Execution and Failure-Preserving Validation

This stage distinguishes reproducible implementation anomalies from generation failures, expected behavior, and spurious discrepancies. Algorithm 1, Lines 16–24, implement execution and failure validation. Lines 17–19 select applicable oracles, execute the test, and detect oracle violations. Lines 20–22 reproduce a violation and reduce the test while preserving the violated relation. Lines 23–24 perform manual validation and retain only reportable bugs.

Oracle selection. Before execution, HistDB selects applicable oracles based on the test semantics and available comparison systems (Algorithm 1, Line 17). Process termination, service loss, and internal errors provide direct failure signals. For tests whose constructs have shared semantics across multiple GDBMSs, HistDB compares normalized execution status and returned rows across the relevant systems. Differential testing is applied only when the compared executions admit a common semantic interpretation; a cross-engine discrepancy alone does not establish a bug.

When an applicable equivalent-query transformation is available, HistDB can also compare the original main query with its generated variant under the same setup and target system. Such a comparison is valid only when the transformation's preconditions hold, including preservation of variable scope, intermediate rows, duplicate multiplicity, ordering, and side effects. The current transformation-based oracle is opportunistic and supports Neo4j/Cypher 25 constructs for which applicable rewrites are available. Table 2 summarizes the representative transformation families and the semantic properties they are intended to preserve. For other result discrepancies, HistDB checks documented language semantics [36] and uses focused controls where appropriate. An execution becomes an anomaly candidate only when an applicable oracle defines an expected relation and the observed behavior violates it.

Table 2. Representative equivalent-query transformation families supported by HistDB for Neo4j/Cypher 25.

Rule	Transformation	Preserved properties
N1: Binding and filtering	Non-aggregating `WITH` projection to `LET`; row filtering with `FILTER`; explicit `CALL` imports	Live bindings, result multiplicity, and null behavior
N2: Sequential and conditional queries	Stage composition with `NEXT`; guarded branches with `WHEN/ELSE`; insertion of an empty `UNION` branch	Intermediate rows, projected columns, duplicate handling, and side effects

Execution and result normalization. Before each test, the target adapter restores a clean graph state. HistDB executes the setup queries in order, followed by the main query, and records execution status, elapsed time, returned columns and rows, and error information. Because database adapters may encode the same logical value differently, HistDB converts returned values into common JSON-like representations and normalizes column names. It retains both order-sensitive and multiset fingerprints: the former preserve row order, while the latter ignore row order but preserve duplicate multiplicity. Timeouts, process termination, and internal errors are also recorded as evidence. This step corresponds to Algorithm 1, Line 18.

Reproduction and failure-preserving reduction. Every initial oracle violation is re-executed from a clean state (Algorithm 1, Line 20). Reproduction succeeds only when the identifying evidence recurs: the same failure signature for a crash or internal error, or the same violated result relation for a differential or equivalent-query oracle. Only reproducible violations proceed to reduction.

HistDB then removes unnecessary setup statements and simplifies the main query's clauses and expressions. Each proposed reduction is re-executed from a clean state and retained only if it produces a smaller test that preserves the same oracle violation. Algorithm 1, Line 22, represents

the resulting reduced test and its execution evidence as (T', E'). This evidence supports subsequent diagnosis and reporting.

Focused controls. When useful, HistDB constructs focused controls by changing one suspected semantic condition at a time. These controls help determine whether the failure depends on a particular interaction rather than on an individual clause in isolation. Comparing these executions with the reduced failure helps localize the interaction responsible for the discrepancy. Focused controls are diagnostic procedures used during investigation, not a mandatory branch for every candidate in Algorithm 1.

Deduplication and reporting. HistDB groups reproducible anomalies by target GDBMS, normalized query structure, observed features, error signature, and minimized failure condition. Analysts manually review each group and merge candidates that expose the same underlying behavior. A bug family is a manually deduplicated group of candidates that reproduce the same minimized failure condition on the same target.

Algorithm 1, Line 23, invokes ManualValidateAndReport on the reduced test and its evidence. This step involves external human review rather than an LLM's unverified judgment. For each reportable finding, analysts prepare a report containing the target version, minimized test, expected and observed behavior, and supporting execution evidence. Line 24 adds a finding to F only when manual validation returns a reportable result.

3.6 Implementation and Reproducibility

HistDB is implemented as a staged Python pipeline that converts historical reports from public issue trackers, regression tests, and developer discussions into structured records and uses them to guide test generation. When a report does not provide executable setup queries, the construction stage attempts to synthesize the setup queries needed by the generated main query, using its dependencies and the selected feature descriptions. The historical corpus is frozen for each evaluation run, and source identifiers preserve the provenance of the knowledge used to construct each test.

Bounded LLM roles. The current implementation invokes an LLM through three schema-constrained calls: skeleton extraction, semantic-feature extraction, and joint construction conditioned on an already selected skeleton–feature composition. When an applicable equivalent-query transformation is available, the construction call also produces a variant of the main query. The model does not select historical reports, seed skeletons, semantic features, comparison systems, oracles, or validation outcomes. Instead, these decisions are handled by explicit pipeline stages, as summarized in Algorithm 1. During reconstruction attempts, the composition target remains fixed; retries address construction failures without changing the selected historical features.

Prompt contracts and reproducibility. Each LLM call has a distinct input–output contract, summarized in Table 3. The extraction calls produce structured, query-grounded representations, whereas the construction call is prompted to instantiate the fixed composition target using retrieved examples and target-system constraints. JSON schemas constrain the returned records and separate machine-readable outputs from free-form model responses. The replication package provides the complete prompt templates and output schemas; the experimental setting fixes the model configuration and other execution settings used in each experiment.

4 Evaluation

4.1 Experimental Setup

We evaluate HistDB by addressing three research questions:

- **RQ1 (Bug Detection):** Can HistDB uncover genuine bugs in Cypher-compatible GDBMSs?

Table 3. Inputs and output contracts of HistDB's schema-constrained LLM calls.

LLM call	Input	Output contract
Skeleton extraction	Historical trigger query	Parameterized skeleton and typed holes; preserve clause order and variable reuse while abstracting application-specific values.
Feature extraction	Historical trigger query	Query-grounded semantic features with canonical names, descriptions, tags/flags, and supporting query snippets.
Joint construction	Fixed composition target, retrieved fragments, and target-system constraints	Ordered setup queries and one main query, with instructions to realize the selected features coherently; an equivalent variant when an applicable transformation exists

- **RQ2 (Historical-knowledge generalization):** To what extent does cross-report selection of historical semantic features lead to generated queries containing query-element combinations absent from the indexed historical corpus?
- **RQ3 (Comparison with existing approaches):** How does HistDB compare with existing graph-database testing approaches in line coverage, query diversity, and validated bug discovery?

Target systems. We evaluate HistDB on five Cypher-compatible GDBMSs: Neo4j, Memgraph, FalkorDB, ArcadeDB, and Apache AGE. These systems include implementations in Java, C++, and C. Table 4 lists the versions covered by the multi-version bug-reporting campaign. The controlled experiments in RQ2 and RQ3 use fixed configurations described in their respective sections.

Experimental setting and environment. For each experiment, we fix the target version, client interface, historical-corpus snapshot, model configuration, random seed, timeout, and execution budget. The historical corpus contains 1,646 issue reports from five graph-database projects, yielding 4,574 skeleton records (3,591 unique skeletons after deduplication) and 18,603 semantic-feature records. Before each test, the target adapter restores a clean database state using a fixed reset procedure. Historical knowledge and retrieval indexes remain unchanged during each run. We compare HistDB with six graph-database test approaches: GDsmith [2], Gamera [4], GDBMeter [3], Dinkel [6], GRev [32], and GraspDB [14].

We conducted our experiments on a virtualized server running 64-bit Ubuntu 24.04.1 LTS, equipped with 48 virtual CPUs (Intel Xeon Gold 5117 at 2.00 GHz) and 125 GiB of RAM. Each tested GDBMS ran in a separate Docker container on the server. HistDB uses DeepSeek-V4.1-Flash for its LLM-based stages, with the model and decoding configuration kept fixed across the controlled experiments.

Table 4. Target GDBMS versions, implementation languages, and bug report statuses as of Sep. 28, 2026.

GDBMS	Campaign versions	Lang	Retained	Confirmed	Fixed
Neo4j	2026.03.0–2026.07.0	Java	14	9	8
Memgraph	3.9.0–3.10.1	C++	26	26	5
FalkorDB	4.16.8–4.18.11	C	40	29	28
ArcadeDB	26.3.2–26.5.1	Java	53	51	50
Apache AGE	1.7.0	C	15	8	8
Total	—	—	**148**	**123**	**99**

4.2 RQ1: Bug Detection

Validation and report status. We audited 161 submitted candidate issues. We attempted to reproduce each candidate from a clean database state. Reproducible failures were minimized and checked against documented language semantics or a justified cross-engine or equivalent-query oracle. We also used focused controls to examine triggering conditions and checked the target's issue tracker for prior reports. We retained 148 reportable bug reports (91.9%). The remaining 13 were excluded because they represented expected behavior, invalid inputs, known duplicates, issues belonging to other projects, results outside the evaluated versions, or unresolved cases.

Table 4 summarizes report status on September 28, 2026. *Retained* counts issue reports that pass manual validation. *Confirmed* requires an explicit developer reproduction, confirmation, acceptance as a bug, or equivalent developer-side fix evidence. *Fixed* requires documented evidence of a correction, such as a developer statement, merged fix, or release containing the fix. Of the 148 retained bug reports, 123 (83.1%) were confirmed by developers, and 99 (66.9%) had documented fixes. By the snapshot date, 80.5% of developer-confirmed bugs had been fixed.

Failure types. Fig. 3 classifies the 148 retained bug reports by observed failure type. A *logic bug* returns an incorrect result; an *internal error* raises an unexpected exception without terminating the service; a *crash* terminates the database process or service; and *other/hang* covers reproducible hangs and failures outside these categories. Overall, 109 bug reports are logic bugs (73.6%), 29 are internal errors, seven are crashes, and three are reproducible hangs or other failures. Logic bugs dominate four targets, while 11 of Neo4j's 14 bugs are internal errors. This distribution shows that HistDB exposes not only visible execution failures but also incorrect results that may otherwise appear plausible.

Bug lifespan. For the 99 bugs fixed in 2026, we traced their earliest verifiable affected releases by testing the corresponding cases across available historical versions. When older releases were unavailable or incompatible, the earliest verifiable affected release provided a conservative bound on the observed lifetime of the bug. Fig. 4 shows the cumulative number of fixed bugs by their earliest verifiable affected year. Among the 99 bugs, 38 were already present before 2026, including 20 that could be traced to 2023 or earlier. The oldest verified case dates to 2021, approximately five calendar years before its fix.

Developer feedback and maintenance impact. Several bug reports prompted developer acknowledgments and regression tests. In ArcadeDB issue #3999 [48], a developer thanked the reporter for the detailed issue, noting that "it's super easy to fix when we have the whole context." ArcadeDB issue #3938 [46] and issue #3957 [47] also received acknowledgments of the reports' detail and the issues' resolution. Beyond individual fixes, ArcadeDB developers added regression tests for ArcadeDB issue #4094 [49] and issue #4095 [50], while FalkorDB added dedicated coverage for issue #1949 [63] through PR #2455 [62] and for related subquery-update behavior through regression test FalkorDB PR #2468 [61]. These cases demonstrate contributions to both defect

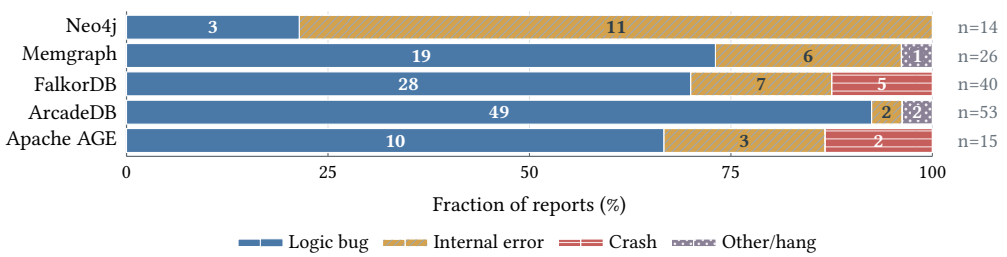


Fig. 3. Distribution of observed failure types among the 148 retained bug reports across five GDBMSs.

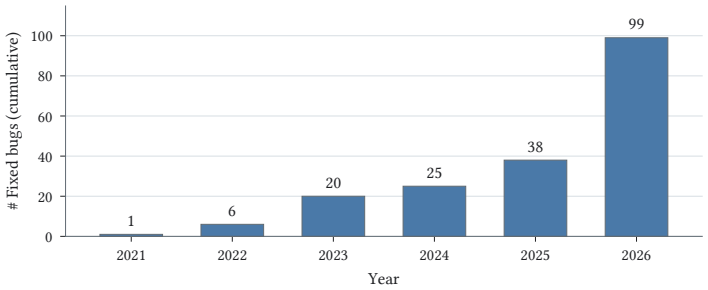


Fig. 4. Cumulative number of bugs fixed in 2026, by earliest verifiable affected year.

correction and regression prevention. Developer rechecks also revealed semantic defects persisting across FalkorDB's legacy C engine and Rust rewrite. In ten FalkorDB issue reports (#1848, #1944, #1946, #1948, #1953, #2000, #2001, #2003, #2006, and #2007) [51–60], both implementations exhibited the reported semantic violations, although their exact outputs sometimes differed. The violations involved relationship uniqueness, row multiplicity, deleted-entity state, null handling, and variable correlation, showing that a substantial rewrite does not necessarily eliminate existing semantic defects. Overall, bug reports across the five GDBMSs suggest that HistDB turns historical failure reports into reusable guidance for uncovering actionable semantic defects.

Answer to RQ1. HistDB yielded 148 validated bug reports across five Cypher-compatible GDBMSs, including 109 logic bugs. Developers confirmed 123 (83.1%), and 99 (66.9%) had documented fixes, 38 of which predated 2026, with the oldest dating to 2021. Several bug reports prompted regression tests, while rechecks revealed semantic defects persisting across FalkorDB's C and Rust implementations. These results demonstrate HistDB's ability to uncover actionable defects and support regression prevention.

4.3 RQ2: Historical-knowledge generalization

We investigate whether cross-report selection of historical semantic features leads to broader exploration of query-element combinations absent from the indexed historical corpus. We compare cross-report composition (CR), which allows features from different historical reports to be selected together, with within-report composition (WR), which selects all features for a composition target from a single historical report. Both settings use the same seed skeletons, select three distinct historical semantic features, and follow the same test-generation process. Thus, the comparison isolates the effect of cross-report feature composition. We set $m = 3$ in Algorithm 1 (Line 6) for the results reported in this section; additional experiments with $m = 4$ and $m = 5$ yielded the same qualitative conclusion and are included in the replication package. We conduct 30 paired runs, retaining 1,000 main queries per setting in each run, for a total of 30,000 queries per setting.

Feature-composition provenance. We first examine whether CR combines historical semantic features beyond the co-occurrence patterns recorded in individual reports. A three-feature selection is considered cross-report if no single historical report contains all three selected feature IDs. A selected feature pair is historically unseen if its two features have never appeared together in any historical report. By construction, WR produces neither cross-report selections nor historically unseen selected feature pairs.

All retained CR selections satisfy the cross-report criterion, and $(99.52 \pm 0.13)\%$ of the selected feature pairs are historically unseen. Among selected pairs whose feature instances originate from different reports, the historically unseen proportion is $(99.95 \pm 0.05)\%$. These results show that

CR primarily combines features beyond the co-occurrence patterns recorded in historical reports. We next examine whether this feature-level composition translates into broader exploration of query-element combinations.

Generation of historically unseen query-element combinations. We assess query-level generalization by comparing the combinations generated by CR and WR against a common historical reference. We apply the same query-element detector to historical reports and generated queries. The detector assigns canonical labels to Cypher constructs or recognized text patterns, such as `optional_match`, `collect`, and `order_by`. Two different detected elements appearing in the same query form an unordered query-element pair.

Let $\mathcal{R}$ denote the indexed historical reports, and let $P(r)$ be the set of distinct query-element pairs detected in report r. We define the historical reference set as

$$H = \bigcup_{r \in \mathcal{R}} P(r). \tag{7}$$

A pair belongs to H if it is detected in at least one indexed historical report under the same detection procedure. For each run, let Q denote its 1,000 generated queries, and let $P(q)$ be the set of distinct query-element pairs detected in query q. We define

$$A(Q) = \bigcup_{q \in Q} P(q), \qquad U(Q) = A(Q) \setminus H. \tag{8}$$

Here, $A(Q)$ contains all distinct query-element pairs generated in the run, whereas $U(Q)$ contains those absent from the historical reference set.

We examine two complementary aspects of query-level generalization: the breadth of historically unseen combinations, measured by $|U(Q)|$, and their coverage across generated queries, measured by the percentage of queries containing at least one pair in $U(Q)$. We also report the total number of distinct generated pairs and the proportion of these pairs that are historically unseen to contextualize the primary results.

Table 5 shows that CR generates 623.47 ± 28.23 distinct query-element pairs per run, compared with 554.47 ± 18.76 for WR. More importantly, CR generates 67.47 ± 13.48 distinct historically unseen pairs, compared with 45.07 ± 6.90 for WR, a 49.7% increase. The proportion of distinct pairs that are historically unseen also rises from (8.11 ± 1.06)% under WR to (10.76 ± 1.75)% under CR. Thus, CR expands the set of generated query-element combinations while increasing the representation of combinations absent from the historical reference.

Coverage of historically unseen combinations across generated queries. The breadth of unseen combinations does not by itself indicate how widely they are distributed across the generated test set. We therefore examine the percentage of queries containing at least one pair in $U(Q)$. Each query is counted once, regardless of how many historically unseen pairs it contains.

As shown in Table 5, this percentage increases from (9.19 ± 0.95)% under WR to (17.81 ± 1.16)% under CR, a gain of 8.62 percentage points. This improvement complements the increase in distinct

Table 5. Query-element generalization under within-report composition (WR) and cross-report composition (CR). Results are reported as mean ± standard deviation over 30 paired runs.

Metric	WR	CR	p-value	Effect size
Distinct pairs	554.47 ± 18.76	623.47 ± 28.23	6.91×10^{-6}	0.873 (L)
Historically unseen pairs	45.07 ± 6.90	67.47 ± 13.48	7.01×10^{-6}	0.847 (L)
Unseen pair ratio (%)	8.11 ± 1.06	10.76 ± 1.75	7.69×10^{-6}	0.817 (L)
Queries with unseen pairs (%)	9.19 ± 0.95	17.81 ± 1.16	6.91×10^{-6}	0.873 (L)

historically unseen pairs: CR not only explores a broader set of previously unobserved combinations but also incorporates such combinations into a larger proportion of its generated queries. The two primary measures therefore capture complementary dimensions of query-level generalization—the breadth of unseen combinations and their coverage across the generated test set.

Statistical evidence. We compare the 30 paired run-level observations using two-sided Wilcoxon signed-rank tests, with Holm correction across the four output measures. All four comparisons remain statistically significant after correction, with adjusted p-values ranging from 6.91×10^{-6} to 7.69×10^{-6}. The corresponding absolute effect sizes, calculated as $|r| = |Z|/\sqrt{30}$, range from 0.817 to 0.873, all indicating large effects (L). Taken together, these results provide consistent statistical evidence that cross-report feature composition enhances query-level generalization, enabling HistDB to generate more distinct query-element pairs absent from the historical corpus and to incorporate such pairs into a larger proportion of generated queries.

Answer to RQ2. Compared with within-report reuse, HistDB's cross-report feature composition generates 49.7% more distinct historically unseen query-element pairs and increases the proportion of queries containing at least one such pair by 8.62 percentage points. These results support cross-report composition as a means of generalizing historical knowledge to query-element combinations not observed in the indexed historical corpus.

4.4 RQ3: Comparison with Existing Approaches

We compare HistDB with six existing graph-database testing approaches along three dimensions: Bug-finding effectiveness, line coverage, and query diversity. We use two distinct experimental protocols. For Bug-finding effectiveness and line coverage, each approach runs for 24 hours, and both outcomes are obtained from the same testing campaign. Bug-finding effectiveness is evaluated across five GDBMSs, whereas line coverage is measured on Neo4j and Memgraph, the two systems that support source-line instrumentation. Query diversity is evaluated separately over 30 independent generation runs of 1,000 main queries per approach. All approaches targeting the same DBMS use the same execution budget and target configuration.

Bug-finding effectiveness. We first compare the distinct validated bugs discovered during the 24-hour testing campaigns. Following the deduplication procedure in Section 3.5, multiple failing queries that reproduce the same underlying defect on the same GDBMS are counted as one bug.

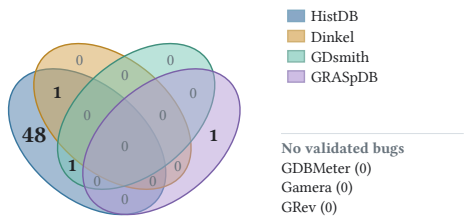


Fig. 5. Overlap in validated bug discoveries by HistDB and six baselines. HistDB uniquely finds 48 bugs, shares one bug each with Dinkel and GDsmith, and misses one additional bug found by GraspDB.

HistDB discovers 50 validated bugs across the five GDBMSs: 3 in Neo4j, 13 in Memgraph, 11 in FalkorDB, 20 in ArcadeDB, and 3 in Apache AGE. These bugs are a subset of the 148 manually validated bug reports shown in RQ1. The six baselines collectively discover three distinct bugs. Dinkel and GDsmith each find one bug also discovered by HistDB, while GraspDB finds one additional bug missed by HistDB. GDBMeter, Gamera, and GRev find no validated bugs in these

campaigns. Thus, the seven approaches collectively discover 51 distinct bugs, of which 48 (96% of the 50 bugs discovered by HistDB in these campaigns) are found only by HistDB. Figure 5 summarizes the overlap.

These results demonstrate that HistDB discovers substantially more validated bugs than the evaluated baselines under the same 24-hour testing budget, while GraspDB contributes one additional discovery that HistDB misses. HistDB therefore achieves a substantially larger discovery yield without subsuming every baseline discovery.

Line coverage. We evaluate Neo4j 2026.03.1 and Memgraph 3.9.0 because both are open source and support source-line instrumentation. We measure Neo4j coverage using JaCoCo [22] over 358,699 executable lines and Memgraph coverage using LCOV/gcov [37, 38] over 130,704 source lines. Coverage is compared only within each DBMS and is collected during the same 24-hour campaigns used for bug discovery.

Figure 6 shows that HistDB achieves the highest observed line coverage on both targets. On Neo4j, HistDB covers 37.60% of the instrumented lines, compared with 26.12% for GraspDB, the strongest baseline. This is an absolute gain of 11.48 percentage points, or a 44.0% relative increase. On Memgraph, HistDB achieves 29.63% coverage, compared with 19.32% for GraspDB, an absolute gain of 10.31 percentage points, or a 53.3% relative increase.

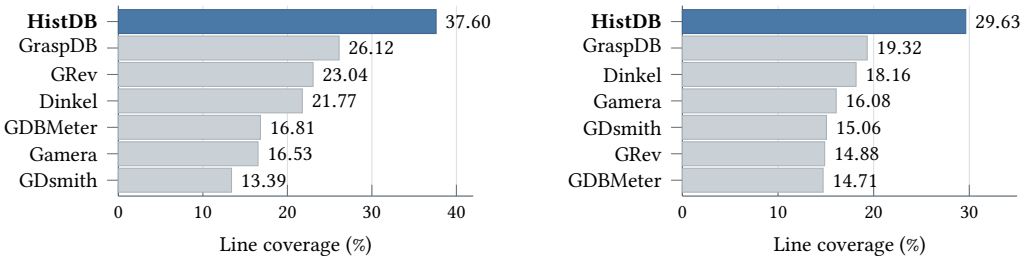
These results show that HistDB exercises more source lines than the evaluated baselines on both instrumented GDBMSs within the same 24-hour budget. Since instrumentation is available only for these two systems in this experiment, the observed coverage advantage should not be generalized to all five GDBMSs.

Query diversity. We separately assess query diversity in terms of normalized query structures and combinations of detected Cypher elements. Each approach is evaluated over 30 independent generation runs of 1,000 main queries each. For each run, we compute four diversity metrics and report their means and sample standard deviations in Table 6.

To derive normalized query shapes, we remove comments and initial numeric Cypher-version declarations, normalize whitespace, and replace identifiers, numeric literals, strings, and parameters with `id`, `num`, `str`, and `param`, respectively. We preserve Cypher keywords, recognized function names, operators, punctuation, and clause order. Consequently, different identifiers are normalized to the same placeholder, and variable identity is not retained.

A *query element* is a canonical label assigned by fixed query-text detection rules to a recognized Cypher construct or textual pattern. For example, `ORDER BY` maps to `order_by`, while `count(...)` produces both `count` and `aggregation`. These labels are used solely to measure diversity and are distinct from HistDB's selected historical semantic features.

We measure four run-level counts: (1) *distinct normalized query shapes*, the number of unique normalized query texts; (2) *distinct query-element sets*, the number of unique sets of detected



(a) Neo4j **(b) Memgraph**

Fig. 6. Line coverage achieved by HistDB and six baselines on Neo4j and Memgraph after 24-hour campaigns.

labels, ignoring label order and repeated occurrences within a query; (3) *distinct query-element pairs*, the number of unique unordered pairs of different labels that co-occur in at least one query; and (4) *distinct core query-element pairs*, the same pair count after excluding eight generic labels: `aggregation`, `match`, `path_pattern`, `property_access`, `read`, `relationship_pattern`, `return`, and `write`. The remaining labels are termed core query elements; for example, `count` remains eligible even when `aggregation` is excluded. Each pair is counted only once per run, irrespective of its frequency. Both pair metrics capture textual co-occurrence, not verified semantic dependencies or bug-triggering interactions.

Table 6 reports the mean and sample standard deviation across the 30 runs. The results show that shape uniqueness alone is insufficient to characterize query diversity. Although GRev and GraspDB achieve statistically higher shape counts than HistDB, their advantage is only about one additional shape per 1,000-query run. HistDB and GraspDB also show no significant difference in the number of distinct query-element sets (adjusted $p = 0.906$, $\delta = -0.019$), suggesting that the main difference lies elsewhere.

Table 6. Query diversity over 30 independent runs of 1,000 queries per approach (mean ± sample SD)

Metric	GDsmith	Gamera	GDBMeter	Dinkel	GRev	GRASpDB	HistDB
Shapes	695.2 ± 15.7	923.7 ± 8.2	898.5 ± 9.8	834.3 ± 10.9	1000.0 ± 0.2	1000.0 ± 0.2	999.0 ± 1.2
Element sets	243.2 ± 8.6	67.4 ± 5.4	57.9 ± 4.1	318.9 ± 9.9	26.2 ± 0.8	881.5 ± 11.4 **(N)**	881.0 ± 8.7
Element pairs	187.9 ± 1.0	175.9 ± 1.2	149.1 ± 1.6	484.7 ± 21.0	55.0 ± 0.0	614.3 ± 14.8	942.0 ± 17.3
Core pairs	79.9 ± 1.0	72.9 ± 1.2	61.1 ± 1.6	289.9 ± 14.1	15.0 ± 0.0	387.2 ± 11.4	636.0 ± 14.9

Legend. Relative to each baseline, light blue means significantly higher HistDB counts, light yellow means significantly lower HistDB counts, and white means no significant difference (two-sided Mann–Whitney U, Holm-adjusted $p < 0.05$). N denotes a negligible effect ($|\delta| < 0.147$); all other baseline comparisons have large effects (L; $|\delta| \geq 0.474$). HistDB is the unshaded reference column.

HistDB's advantage instead lies in pairwise co-occurrence diversity. Compared with GraspDB, the strongest baseline on both pair metrics, HistDB covers 53.4% more distinct query-element pairs and 64.3% more core query-element pairs. Both metrics exhibit large effects against all six baselines ($\delta = 1$; adjusted $p \leq 4.52 \times 10^{-10}$). A Cliff's δ of 1 indicates complete separation of the observed samples: every HistDB run has a higher pair count than every run of each baseline.

This advantage persists after generic elements such as `MATCH` or `RETURN` are excluded, indicating that it is not merely driven by common query constructs. Overall, HistDB's diversity advantage lies primarily in the breadth of detected element co-occurrences. By generating queries with a wider range of combinations of Cypher constructs, HistDB may exercise a broader range of database behaviors, creating more opportunities to expose defects missed by existing approaches.

Answer to RQ3. HistDB identifies 50 validated bugs, 48 of which are not found by any baseline, and achieves the highest observed line coverage on both instrumented GDBMSs. It also covers 53.4% more query-element pairs and 64.3% more core pairs than the strongest baseline, although GraspDB finds one additional bug missed by HistDB.

4.5 Representative Bug Cases

Fig. 7 presents five minimized bug-triggering queries, one from each evaluated GDBMS. Each case includes the expected and observed behavior, together with focused controls that help isolate the triggering conditions.

Neo4j: planner verification failure. In Neo4j 2026.05.0 issue #13827 [41], a query combining `NEXT`, `UNION`, `FILTER false`, and a subsequent `UNWIND []` fails during planning, even under `EXPLAIN`. Removing `NEXT`, removing either `FILTER false` or the following `UNWIND`, or replacing `FILTER false` with

```
CYPHER 25
RETURN 1 AS y
NEXT
RETURN 1 AS y
UNION
FILTER false
UNWIND [] AS z
RETURN z AS y;
```
Expected: `y = 1`
Observed: `internal planner error` ✗

(a) Neo4j `issue #13827`: planner failure.

```
CREATE (a:Person {name:'Alice'}),
       (b:Person {name:'Bob'}),
       (a)-[:KNOWS]->(b);
MATCH (a:Person {name:'Alice'})
      -[r:KNOWS]->
      (b:Person {name:'Bob'})
WITH a, b, r
MATCH p = (a)-[:KNOWS*]->(b)
RETURN count(r) AS rc;
```
Expected: `rc = 1`
Observed: `rc = 0` ✗

(b) ArcadeDB `issue #3999`: binding loss.

```
CREATE (:Person {name:'Alice'}),
       (:Person {name:'Bob'}),
       (:Person {name:'Charlie'})
MERGE (e:Person)
MERGE (:Person {name:'Eve'})
RETURN e.name AS eName
ORDER BY eName;
```
Expected: `Alice, Bob, Charlie`
Observed: `Alice, Bob, Charlie, Eve` ✗

(c) Memgraph `issue #4162`: `MERGE` binding.

```
UNWIND [null] AS x
RETURN count(x) AS c;
```
Expected: `c = 0`
Observed: `c = 1` ✗

(d) Apache AGE `issue #2383`: null counting.

```
CREATE (a:Person {name:'Alice'}),
       (b:Person {name:'Bob'})
WITH a, b
MATCH (a), (b)
RETURN a.name AS a, b.name AS b;
```
Expected: `Alice, Bob`
Observed: `server crash, exit 139` ✗

(e) FalkorDB `issue #1949`: rematching crash.

Fig. 7. Representative minimized bug-triggering queries from five Cypher-compatible GDBMSs, with expected and observed behavior shown for each case.

`WITH`/`WHERE false` avoids the failure. These controls suggest that the planner error depends on the interaction among these constructs in the empty UNION branch.

ArcadeDB: lost relationship binding. In ArcadeDB 26.4.2 issue #3999 [48], an initial `MATCH` binds relationship variable `r`, which is carried through `WITH`. A subsequent variable-length path match over the same endpoints causes `count(r)` to return zero. Replacing the second match with a fixed-length pattern restores the expected result, whereas the explicitly bounded form `*1..1` still triggers the failure. This indicates that the discrepancy is associated with variable-length pattern handling rather than the number of hops traversed.

Memgraph: retroactive expansion of a `MERGE` binding. In Memgraph 3.9.0 issue #4162 [42], `MERGE (e:Person)` binds the three existing `Person` nodes, but a subsequent same-label `MERGE` unexpectedly causes the newly created node `Eve` to appear in the earlier binding's results. Removing the later `MERGE`, changing the new node's label, or inserting a `WITH e` boundary restores the expected results. These controls indicate that the discrepancy depends on the interaction between the earlier label-only binding and the later same-label write within the same query pipeline.

Apache AGE: incorrect null aggregation. In Apache AGE 1.7.0 issue #2383 [64], `UNWIND [null] AS x RETURN count(x)` returns one instead of zero. Direct controls using `count(null)` and `WITH null AS x` return zero, while directly returning `x` after `UNWIND` confirms that the produced value is null. These controls indicate that the discrepancy arises from the interaction between `UNWIND` and aggregation.

FalkorDB: crash on rematching bound variables. In FalkorDB 4.18.1 issue #1949 [63], a query carries two node bindings through `WITH` and then rematches both variables in a single `MATCH`, causing the server to terminate with exit code 139. Rematching either variable individually succeeds, and the failure also occurs when the bindings originate from an earlier `MATCH` rather than `CREATE`. These controls suggest that the failure is associated with simultaneously rematching multiple previously bound variables.

Together, these cases illustrate the diversity of implementation defects detected across the five GDBMSs. Focused controls contrast each failure-triggering query with closely related variants that execute correctly, helping isolate the triggering conditions, while minimized reproducers make the failures compact and inspectable. These cases complement the aggregate RQ1 results by illustrating the range of defects HistDB detects and the query conditions under which they manifest.

5 Discussion

In this section, we first discuss the implications of our results for research and practice. Then, we examine the threats to the validity of our study.

5.1 Implications for Research and Practice

Our findings have implications for history-guided testing, its integration with existing techniques, and practical database maintenance.

Compositional reuse of historical failures. Historical bug reports can serve not only as regression tests but also as structured guidance for generating new tests. Separating structure-preserving seed skeletons from historically observed, source-linked semantic features enables query behaviors extracted from different reports to be recombined while preserving execution-critical context. This suggests a general principle for history-guided testing: preserve the structure and dependencies needed to realize a behavior while recombining mechanisms implicated in past failures.

Complementary testing strategies. HistDB guides test construction rather than introducing a new correctness oracle. It can therefore complement grammar- and state-aware generation, differential testing, and metamorphic testing: historical knowledge guides what to explore, generators construct executable tests, and applicable oracles identify anomalies. This separation integrates historical guidance into existing testing pipelines without replacing their generation or validation components, while preserving existing testing workflows.

Practical database maintenance. Minimized queries, setup statements, and execution evidence help developers diagnose and repair defects. In ArcadeDB issue #3999 [48], a developer noted that the detailed context made the issue easier to fix. Bug reports also prompted regression tests in ArcadeDB issue #4094 [49] and issue #4095 [50] and FalkorDB issue #1949 [63], extending their impact from defect correction to recurrence prevention. Moreover, semantic violations persisted across FalkorDB's legacy C engine and Rust rewrite. Although these cases do not establish how frequently rewrites preserve defects, they show that rewriting an implementation does not guarantee semantic correctness. Preserving historical triggers, expected behavior, and provenance can therefore support regression checks as implementations evolve.

5.2 Threats to Validity

Construct validity. RQ2 examines selected-feature provenance and query-element pairs observed in generated queries as different aspects of composition. We use the same detector for historical and generated queries. A detected pair indicates that two elements appear in the same query, but does not prove that they caused a bug. RQ2 statistics are based on 30 paired WR/CR groups, each containing 1,000 main queries per setting. RQ3 measures query diversity using normalized query text and fixed query-element labels.

Internal validity. A key threat is whether the observed results are affected by implementation settings or historical-corpus leakage. We mitigate these threats by fixing DBMS images, drivers, configurations, historical-corpus and retrieval snapshots, and generation seeds, and by executing tests from clean database states. Furthermore, each evaluation campaign uses a frozen historical corpus, preventing newly discovered bug reports from entering the corpus used for test generation in that campaign. For RQ2, W and S use the same historical corpus, LLM settings, seed query skeletons,

and query-element detector. We evaluate them over 30 paired groups, each containing 1,000 main queries per setting, and apply paired statistical tests to account for the matched observations.

External validity. Our evaluation covers five independently implemented Cypher-compatible graph database management systems (GDBMSs), providing evidence across different database implementations. However, the generalizability of HistDB to other graph query languages and projects with sparse or less informative historical bug reports remains to be investigated.

6 Related Work

Graph-database testing. Automated DBMS testing combines input generation with behavioral oracles, including differential testing, query partitioning, and metamorphic testing [12, 13, 23–26]. Existing graph-database testers generate Cypher queries using query structure and graph state, or derive follow-up tests through predefined transformations and relations. For example, GDsmith generates Cypher queries, while Dinkel tracks query context and graph schema to generate state-aware queries and applies semantics-preserving query transformations [2, 6]. Other approaches employ query partitioning (GDBMeter), graph-aware metamorphic relations (Gamera), injective and surjective graph-query transformations (GraphGenie), equivalent-query rewriting (GRev), or graph-state persistence checking (GraspDB) [3–5, 14, 32]. In contrast to these approaches, HistDB mines historical Cypher triggers into seed skeletons and reusable semantic features, then recombines them to generate tests while leveraging existing oracles for bug detection.

LLM-assisted test generation. Large language models (LLMs) have been used for direct test synthesis, test improvement, and feedback-guided generation [15–17, 27–31]. Retrieval- augmented approaches further use documentation, examples, or reference tests to provide contextual knowledge for test generation [10, 11, 18]. In graph-database testing, DGDB uses LLMs to generate Cypher and Gremlin queries and applies differential testing to detect inconsistent results [7]. HistDB also uses an LLM and retrieved context, but fixes the generation target before construction. It combines a seed skeleton with semantic features selected from historical reports, then prompts the LLM to construct setup queries and a main query conditioned on that target.

Historical-bug-guided test generation. Prior work has used historical failures to configure test generators, recover real programs or reusable test ingredients, and derive generation structures from failure-triggering inputs [19–21, 33, 34]. BugForge is a closely related DBMS-testing framework that constructs a repository of historical bug reports, extracts proof-of-concept (PoC) inputs, and semantically adapts them for fuzzing, regression testing, and cross-DBMS bug discovery [35]. Whereas BugForge centers on extracting and adapting historical PoCs, HistDB decomposes historical Cypher triggers into seed skeletons and reusable semantic features, selects features across reports, and jointly constructs setup queries and main queries to explore feature combinations that may not occur together in any individual historical trigger.

7 Conclusion

In this paper, we investigate whether historical bug reports can provide reusable knowledge for generating new tests for graph database systems. We propose HistDB, an approach that decomposes historical Cypher triggers into seed skeletons and semantic features, selects features across reports according to their structural support in a seed skeleton, and prompts an LLM to jointly construct setup and main queries to explore new combinations of historical query elements. Our evaluation across five Cypher-compatible GDBMSs demonstrates that HistDB effectively generalizes historical knowledge to previously unseen query-element combinations, discovers actionable bugs, and improves query diversity and line coverage over existing testing approaches. These results highlight the potential of historical failures as a source of compositional testing knowledge that extends beyond reproducing known bugs.

Data-Availability Statement

An anonymized preliminary artifact is available at https://anonymous.4open.science/r/HistDB-C02C/. It includes the HistDB implementation, experimental configurations, and scripts for reproducing the evaluation. Full reproduction requires access to the evaluated GDBMS versions, the specified LLM, and sufficient computational resources. We plan to submit the finalized artifact for Artifact Evaluation.

References

[1] N. Francis *et al.*, "Cypher: An evolving query language for property graphs," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2018, pp. 1433–1445. doi: 10.1145/3183713.3190657.

[2] Z. Hua *et al.*, "GDsmith: Detecting bugs in Cypher graph database engines," in *Proc. ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2023, pp. 163–174. doi: 10.1145/3597926.3598046.

[3] M. Kamm, M. Rigger, C. Zhang, and Z. Su, "Testing graph database engines via query partitioning," in *Proc. ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2023, pp. 140–149. doi: 10.1145/3597926.3598044.

[4] Z. Zhuang, P. Li, P. Ma, W. Meng, and S. Wang, "Testing graph database systems via graph-aware metamorphic relations," *Proc. VLDB Endowment*, vol. 17, no. 4, pp. 836–848, 2023. doi: 10.14778/3636218.3636236.

[5] Y. Jiang, J. Liu, J. Ba, R. H. C. Yap, Z. Liang, and M. Rigger, "Detecting logic bugs in graph database management systems via injective and surjective graph query transformation," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2024. doi: 10.1145/3597503.3623307.

[6] C. Wüst, Z. Jiang, and Z. Su, "Dinkel: State-aware and granular framework for validating graph databases," *Proc. VLDB Endowment*, vol. 19, no. 6, pp. 1198–1211, 2026. doi: 10.14778/3797919.3797928.

[7] J. Wu, Z. Wu, R. Li, H. Qin, and G. Wang, "Effective bug detection in graph database engines: An LLM-based approach," *CoRR*, abs/2402.00292, 2024. doi: 10.48550/arXiv.2402.00292.

[8] N. Francis *et al.*, "Formal semantics of the language Cypher," *CoRR*, abs/1802.09984, 2018. doi: 10.48550/arXiv.1802.09984.

[9] A. Green *et al.*, "Updating graph databases with Cypher," *Proc. VLDB Endowment*, vol. 12, no. 12, pp. 2242–2253, 2019. doi: 10.14778/3352063.3352139.

[10] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[11] J. Shin, N. Shiri Harzevili, R. Aleithan, H. Hemmati, and S. Wang, "Retrieval-augmented test generation: How far are we?" in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2026. Author preprint: arXiv:2409.12682.

[12] M. Rigger and Z. Su, "Testing database engines via pivoted query synthesis," in *Proc. 14th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2020, pp. 667–682.

[13] M. Rigger and Z. Su, "Finding bugs in database systems via query partitioning," *Proc. ACM Programming Languages*, vol. 4, OOPSLA, Art. 211, 2020. doi: 10.1145/3428279.

[14] S. Liu *et al.*, "Testing graph database systems with graph-state persistence oracle," in *Proc. ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2024, pp. 666–677. doi: 10.1145/3650212.3680311.

[15] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Trans. Software Engineering*, vol. 50, no. 4, pp. 911–936, 2024. doi: 10.1109/TSE.2024.3368208.

[16] N. Alshahwan *et al.*, "Automated unit test improvement using large language models at Meta," in *Companion Proc. ACM Int. Conf. Foundations of Software Engineering (FSE)*, 2024, pp. 185–196. doi: 10.1145/3663529.3663839.

[17] Y. Li, P. Liu, H. Wang, J. Chu, and W. E. Wong, "Evaluating large language models for software testing," *Computer Standards & Interfaces*, vol. 93, Art. 103942, 2025. doi: 10.1016/j.csi.2024.103942.

[18] Z. Zhang, X. Liu, Y. Lin, X. Gao, H. Sun, and Y. Yuan, "Reference-based retrieval-augmented unit test generation," *ACM Trans. Software Engineering and Methodology*, vol. 35, no. 6, Art. 149, 2026. doi: 10.1145/3765758.

[19] M. R. I. Rabin and M. A. Alipour, "Configuring test generators using bug reports: A case study of GCC compiler and Csmith," in *Proc. ACM/SIGAPP Symp. Applied Computing (SAC)*, 2021, pp. 1750–1758. doi: 10.1145/3412841.3442047.

[20] H. Zhong, "Enriching compiler testing with real program from bug report," in *Proc. IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2022, Art. 40, 12 pages. doi: 10.1145/3551349.3556894.

[21] H. Wang, Z. Xu, and S. H. Tan, "Testing refactoring engine via historical bug report driven LLM," in *Proc. IEEE/ACM Int. Conf. AI Foundation Models and Software Engineering (Forge)*, 2025, pp. 113–124. doi: 10.1109/FORGE66646.2025.00020.

[22] JaCoCo contributors, "JaCoCo Java code coverage library." [Online]. Available: https://www.jacoco.org/jacoco/. Accessed: Sep. 7, 2026.

[23] W. M. McKeeman, "Differential testing for software," *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.

[24] T. Y. Chen, S. C. Cheung, and S. M. Yiu, "Metamorphic testing: A new approach for generating next test cases," Dept. Computer Science, Hong Kong Univ. of Science and Technology, Tech. Rep. HKUST-CS98-01, 1998.

[25] S. Segura, G. Fraser, A. B. Sánchez, and A. Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Trans. Software Engineering*, vol. 42, no. 9, pp. 805–824, 2016. doi: 10.1109/TSE.2016.2532875.

[26] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys*, vol. 51, no. 1, Art. 4, 2018. doi: 10.1145/3143561.

[27] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "CodaMOSA: Escaping coverage plateaus in test generation with pre-trained large language models," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2023, pp. 919–931. doi:

10.1109/ICSE48619.2023.00085.
[28] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Trans. Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024. doi: 10.1109/TSE.2023.3334955.
[29] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proc. ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2023, pp. 423–435. doi: 10.1145/3597926.3598067.
[30] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, and L. Zhang, "Fuzz4All: Universal fuzzing with large language models," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2024, pp. 1547–1559. doi: 10.1145/3597503.3639121.
[31] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, "ChatUniTest: A framework for LLM-based test generation," in *Companion Proc. ACM Int. Conf. Foundations of Software Engineering (FSE)*, 2024, pp. 572–576. doi: 10.1145/3663529.3663801.
[32] Q. Mang, A. Fang, B. Yu, H. Chen, and P. He, "Testing graph database systems via equivalent query rewriting," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2024, 12 pages. doi: 10.1145/3597503.3639200.
[33] M. Sun, Y. Yang, M. Wen, Y. Wang, Y. Zhou, and H. Jin, "Validating SMT solvers via skeleton enumeration empowered by historical bug-triggering inputs," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2023, pp. 69–81. doi: 10.1109/ICSE48619.2023.00018.
[34] Y. Zhao, Z. Wang, J. Chen, M. Liu, M. Wu, Y. Zhang, and L. Zhang, "History-driven test program synthesis for JVM testing," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2022, pp. 1133–1144. doi: 10.1145/3510003.3510059.
[35] D. Li, Q. Liu, Y. Guo, J. Liang, Z. Wu, C. Zhang, J. Fu, H. Mao, Z. Guan, and Y. Jiang, "BugForge: Constructing and utilizing DBMS bug repository to enhance DBMS testing," *arXiv preprint arXiv:2604.03024*, 2026. doi: 10.48550/arXiv.2604.03024.
[36] Neo4j, Inc., "Cypher Manual." [Online]. Available: https://neo4j.com/docs/cypher-manual/current/. Accessed: Sep. 7, 2026.
[37] Linux Test Project, "LCOV code coverage tool." [Online]. Available: https://github.com/linux-test-project/lcov. Accessed: Sep. 7, 2026.
[38] Free Software Foundation, "gcov—A test coverage program," in *Using the GNU Compiler Collection (GCC)*. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Gcov.html. Accessed: Sep. 7, 2026.
[39] Neo4j. 2021. Collect in a simple query fails: Should not reach await until tracking is complete, cancelled or out-of demand. GitHub issue #12657. Retrieved October 14, 2026 from https://github.com/neo4j/neo4j/issues/12657.
[40] Neo4j. 2023. Unexpected Error about "Variable not defined". GitHub issue #13221. Retrieved October 14, 2026 from https://github.com/neo4j/neo4j/issues/13221.
[41] Neo4j. 2026. Planner internal error with NEXT followed by UNION branch using FILTER false before UNWIND. GitHub issue #13827. Retrieved October 14, 2026 from https://github.com/neo4j/neo4j/issues/13827.
[42] Memgraph. 2026. Label-only MERGE binding may expand after a later same-label MERGE unless separated by WITH. GitHub issue #4162. Retrieved October 14, 2026 from https://github.com/memgraph/memgraph/issues/4162.
[43] Memgraph. 2026. OPTIONAL MATCH + collect(...) followed by ORDER BY size([pattern comprehension]) may trigger internal "expected a list, got null" behavior. GitHub issue #4180. Retrieved October 14, 2026 from https://github.com/memgraph/memgraph/issues/4180.
[44] Memgraph. 2026. fix(query): correlate pattern comprehensions and scope CALL subquery variables. GitHub pull request #4487. Retrieved October 14, 2026 from https://github.com/memgraph/memgraph/pull/4487.
[45] ArcadeDB. 2026. Bolt: Parameterized queries fail to match in MATCH property maps. GitHub issue #3560. Retrieved October 14, 2026 from https://github.com/ArcadeData/arcadedb/issues/3560.
[46] ArcadeDB. 2026. An existential pattern predicate in WHERE may fail to filter rows by the target node pattern. GitHub issue #3938. Retrieved October 14, 2026 from https://github.com/ArcadeData/arcadedb/issues/3938.
[47] ArcadeDB. 2026. COLLECT subqueries may return null instead of the collected list. GitHub issue #3957. Retrieved October 14, 2026 from https://github.com/ArcadeData/arcadedb/issues/3957.
[48] ArcadeDB. 2026. Relationship variable may be lost after a later variable-length match. GitHub issue #3999. Retrieved October 14, 2026 from https://github.com/ArcadeData/arcadedb/issues/3999.
[49] ArcadeDB. 2026. CALL db.labels() YIELD label may nullify carried variables. GitHub issue #4094. Retrieved October 14, 2026 from https://github.com/ArcadeData/arcadedb/issues/4094.
[50] ArcadeDB. 2026. Consecutive directed relationship patterns may reuse the same relationship. GitHub issue #4095. Retrieved October 14, 2026 from https://github.com/ArcadeData/arcadedb/issues/4095.
[51] FalkorDB. 2026. count(*) may undercount parallel edges of different relationship types between the same endpoints. GitHub issue #1848. Retrieved October 14, 2026 from https://github.com/FalkorDB/FalkorDB/issues/1848.
[52] FalkorDB. 2026. Converging pattern may reuse the same relationship twice and fabricate extra matches. GitHub issue #1944. Retrieved October 14, 2026 from https://github.com/FalkorDB/FalkorDB/issues/1944.
[53] FalkorDB. 2026. Deleted node values may retain pre-delete properties when returned after DELETE. GitHub issue #1946. Retrieved October 14, 2026 from https://github.com/FalkorDB/FalkorDB/issues/1946.
[54] FalkorDB. 2026. Grouped aggregation may collapse parallel relationships between the same endpoints. GitHub issue #1948. Retrieved October 14, 2026 from https://github.com/FalkorDB/FalkorDB/issues/1948.
[55] FalkorDB. 2026. Pattern comprehension may drop matched null projection elements instead of returning [null]. GitHub issue #1953. Retrieved October 14, 2026 from https://github.com/FalkorDB/FalkorDB/issues/1953.
[56] FalkorDB. 2026. Relationship variable carried through WITH may lose identity in a later MATCH. GitHub issue #2000. Retrieved October 14, 2026 from https://github.com/FalkorDB/FalkorDB/issues/2000.

[57] FalkorDB. 2026. Duration ordering comparisons may return boolean instead of null. GitHub issue #2001. Retrieved October 14, 2026 from https://github.com/FalkorDB/FalkorDB/issues/2001.
[58] FalkorDB. 2026. List comprehension over collect(...) may return null when the expression uses an outer variable. GitHub issue #2003. Retrieved October 14, 2026 from https://github.com/FalkorDB/FalkorDB/issues/2003.
[59] FalkorDB. 2026. Nested pattern comprehension inside a list comprehension may ignore the outer variable. GitHub issue #2006. Retrieved October 14, 2026 from https://github.com/FalkorDB/FalkorDB/issues/2006.
[60] FalkorDB. 2026. Anonymous undirected relationship pattern may collapse parallel edges between the same endpoints. GitHub issue #2007. Retrieved October 14, 2026 from https://github.com/FalkorDB/FalkorDB/issues/2007.
[61] FalkorDB. 2026. test: writes inside a subquery apply to every input row. GitHub pull request #2468. Retrieved October 14, 2026 from https://github.com/FalkorDB/FalkorDB/pull/2468.
[62] FalkorDB. 2026. test(with): cover re-MATCHing multiple variables carried through WITH (#1949). GitHub pull request #2455. Retrieved October 14, 2026 from https://github.com/FalkorDB/FalkorDB/pull/2455.
[63] FalkorDB. 2026. MATCH on multiple carried variables after WITH crashes the server. GitHub issue #1949. Retrieved October 14, 2026 from https://github.com/FalkorDB/FalkorDB/issues/1949.
[64] Apache AGE. 2026. UNWIND [null] may produce a row whose value prints as null, but count(x) still counts it as 1. GitHub issue #2383. Retrieved October 14, 2026 from https://github.com/apache/age/issues/2383.
[65] RedisGraph. 2022. Indexing not used in many cases, why? Documentation update? GitHub issue #2386. Retrieved October 14, 2026 from https://github.com/RedisGraph/RedisGraph/issues/2386.